

Comandos Básicos en Python

input, print y operaciones con variables enteras

Documento elaborado para el curso de Programación

22 de julio de 2026

Índice

1. Introducción	2
2. El comando input()	2
2.1. ¿Qué es input()?	2
2.2. Sintaxis básica	2
2.3. Ejemplos de uso	2
2.4. Importante: input() siempre devuelve texto	3
3. El comando int()	3
3.1. ¿Qué es int()?	3
3.2. Sintaxis básica	3
3.3. Ejemplos de conversión	3
3.4. Combinando input() y int()	3
3.5. Manejo de errores con int()	4
4. El comando print()	4
4.1. ¿Qué es print()?	4
4.2. Sintaxis básica	4
4.3. Parámetros importantes de print()	4
4.3.1. separador (sep)	4
4.3.2. final (end)	4
4.4. Ejemplos de uso de print()	5
5. Operaciones con variables numéricas enteras	5
5.1. Declaración de variables enteras	5
5.2. Operadores aritméticos básicos	5
5.3. Ejemplos de operaciones	5
5.4. Operadores de asignación	6
6. Programas completos de ejemplo	6
6.1. Ejemplo 1: Calculadora simple	6
6.2. Ejemplo 2: Conversión de unidades	7
6.3. Ejemplo 3: Promedio de calificaciones	7
7. Resumen y recomendaciones	7
7.1. Puntos clave	7
7.2. Errores comunes a evitar	8
8. Conclusión	8

1 Introducción

Python es un lenguaje de programación de alto nivel que se caracteriza por su sintaxis clara y legible. En este documento, exploraremos tres elementos fundamentales para cualquier programa en Python:

- El comando `input()` para la entrada de datos
- El comando `print()` para la salida de información
- Las operaciones con variables numéricas enteras
- El comando `int()` para conversión de tipos

Estos elementos constituyen la base para interactuar con el usuario y realizar cálculos básicos.

2 El comando `input()`

2.1 ¿Qué es `input()`?

El comando `input()` en Python permite recibir información del usuario a través del teclado. Su funcionamiento es simple: cuando el programa llega a esta instrucción, se detiene y espera a que el usuario escriba algo y presione Enter.

2.2 Sintaxis básica

La sintaxis básica del comando es:

```
1 variable = input("Mensaje para el usuario: ")
```

El texto dentro de los paréntesis es opcional y sirve como mensaje informativo para el usuario.

2.3 Ejemplos de uso

```
1 # Ejemplo 1: Solicitar un nombre
2 nombre = input(" Cmo te llamas? ")
3 print("Hola", nombre)
4
5 # Ejemplo 2: Solicitar edad
6 edad = input(" Cuntos a os tienes? ")
7 print("Tienes", edad, "a os")
8
9 # Ejemplo 3: M ltiples preguntas
10 pais = input(" De qu pa s eres? ")
11 ciudad = input(" En qu ciudad vives? ")
12 print("Vives en", ciudad + ",", pais)
```

2.4 Importante: input() siempre devuelve texto

Es fundamental recordar que `input()` siempre devuelve una cadena de texto (*string*), incluso cuando el usuario ingresa números. Por ejemplo:

```
1 numero = input("Ingresa un número: ")
2 print(type(numero)) # Salida: <class 'str'>
```

Esto significa que si intentamos hacer operaciones matemáticas directamente con el valor ingresado, obtendremos errores.

3 El comando int()

3.1 ¿Qué es int()?

El comando `int()` es una función incorporada en Python que convierte un valor a un número entero. Esta función es esencial cuando trabajamos con datos ingresados por el usuario mediante `input()`.

3.2 Sintaxis básica

```
1 numero_entero = int(valor_a_convertir)
```

3.3 Ejemplos de conversión

```
1 # Convertir texto a entero
2 texto = "123"
3 numero = int(texto)
4 print(numero) # 123
5 print(type(numero)) # <class 'int'>
6
7 # Convertir número decimal a entero (trunca la parte decimal)
8 decimal = 3.14
9 entero = int(decimal)
10 print(entero) # 3
11
12 # Convertir input a entero
13 edad_texto = input("Ingresa tu edad: ")
14 edad = int(edad_texto)
15 print("El año que viene tendrás", edad + 1, "años")
```

3.4 Combinando input() y int()

La combinación más común es recibir datos del usuario y convertirlos a enteros para realizar operaciones:

```
1 # Solicitar un número y convertirlo a entero
2 numero1 = int(input("Ingresa el primer número: "))
3 numero2 = int(input("Ingresa el segundo número: "))
```

```

4
5 # Realizar operaciones
6 suma = numero1 + numero2
7 print("La suma es:", suma)

```

3.5 Manejo de errores con int()

Es importante tener en cuenta que `int()` puede generar errores si intentamos convertir un valor que no es numérico:

```

1 # Esto generar un error (ValueError)
2 # numero = int("hola")

```

Para evitar estos errores, podemos usar estructuras de control como `try` y `except`.

4 El comando print()

4.1 ¿Qué es print()?

El comando `print()` es la forma más básica de mostrar información en Python. Permite enviar texto, números y otros tipos de datos a la consola o salida estándar.

4.2 Sintaxis básica

```

1 print("Texto a mostrar")
2 print(variable)
3 print("Texto", variable, "m s texto")

```

4.3 Parámetros importantes de print()

4.3.1. separador (sep)

Permite especificar qué carácter se utiliza para separar los elementos:

```

1 nombre = "Ana"
2 edad = 25
3 print(nombre, edad, sep=" - ") # Ana - 25
4 print(1, 2, 3, sep=",") # 1,2,3

```

4.3.2. final (end)

Permite especificar qué carácter se coloca al final de la línea:

```

1 print("Hola", end=" ")
2 print("mundo") # Hola mundo
3
4 print("Primero", end=", ")
5 print("segundo") # Primero, segundo

```

4.4 Ejemplos de uso de print()

```
1 # Mostrar texto literal
2 print("Bienvenido al programa")
3
4 # Mostrar variables
5 nombre = "Carlos"
6 edad = 30
7 print("Nombre:", nombre)
8 print("Edad:", edad)
9
10 # Mostrar mltiples elementos
11 print("Nombre:", nombre, "Edad:", edad)
12
13 # Mostrar resultados de operaciones
14 resultado = 15 + 20
15 print("El resultado es:", resultado)
16
17 # Formateo de salida
18 print(f"Nombre: {nombre}, Edad: {edad}")
19 print("Nombre: {}, Edad: {}".format(nombre, edad))
```

5 Operaciones con variables numéricas enteras

5.1 Declaración de variables enteras

En Python, no es necesario declarar el tipo de variable; el intérprete lo infiere automáticamente:

```
1 # Asignaci n directa
2 edad = 25
3 cantidad = 100
4 temperatura = -5
5
6 # Desde input (con conversi n)
7 numero = int(input("Ingresa un n mero: "))
```

5.2 Operadores aritméticos básicos

Python soporta los siguientes operadores para números enteros:

5.3 Ejemplos de operaciones

```
1 # Variables enteras
2 a = 10
3 b = 3
4
5 # Operaciones b sicas
6 suma = a + b
```

Operador	Descripción	Ejemplo
+	Suma	$5 + 3 = 8$
-	Resta	$5 - 3 = 2$
*	Multiplicación	$5 * 3 = 15$
/	División (flotante)	$5 / 3 = 1.666...$
//	División entera	$5 // 3 = 1$
%	Módulo (resto)	$5 \% 3 = 2$
**	Potencia	$5 ** 3 = 125$

Cuadro 1: Operadores aritméticos en Python

```

7 resta = a - b
8 multiplicacion = a * b
9 division = a / b          # Divisi n flotante
10 division_entera = a // b  # Divisi n entera
11 modulo = a % b
12 potencia = a ** b
13
14 print(f"Suma: {suma}")
15 print(f"Resta: {resta}")
16 print(f"Multiplicaci n: {multiplicacion}")
17 print(f"Divisi n flotante: {division}")
18 print(f"Divisi n entera: {division_entera}")
19 print(f"M dulo: {modulo}")
20 print(f"Potencia: {potencia}")

```

5.4 Operadores de asignación

Python también ofrece operadores de asignación que combinan una operación con la asignación:

```

1 x = 5
2 x += 3    # Equivale a x = x + 3  -> x = 8
3 x -= 2    # Equivale a x = x - 2  -> x = 6
4 x *= 4    # Equivale a x = x * 4   -> x = 24
5 x /= 3    # Equivale a x = x / 3   -> x = 8
6 x %= 3    # Equivale a x = x % 3   -> x = 2
7 x **= 2   # Equivale a x = x ** 2  -> x = 4

```

6 Programas completos de ejemplo

6.1 Ejemplo 1: Calculadora simple

```

1 # Calculadora de dos n meros
2 print("=== CALCULADORA SIMPLE ===")
3
4 num1 = int(input("Ingresa el primer n mero: "))
5 num2 = int(input("Ingresa el segundo n mero: "))

```

```

6
7 print("\nResultados:")
8 print(f"Suma: {num1} + {num2} = {num1 + num2}")
9 print(f"Resta: {num1} - {num2} = {num1 - num2}")
10 print(f"Multiplicaci n: {num1} * {num2} = {num1 * num2}")
11 print(f"Divisi n exacta: {num1} / {num2} = {num1 / num2}")
12 print(f"Divisi n entera: {num1} // {num2} = {num1 // num2}")
13 print(f"Resto: {num1} % {num2} = {num1 % num2}")
14 print(f"Potencia: {num1} ** {num2} = {num1 ** num2}")

```

6.2 Ejemplo 2: Conversi3n de unidades

```

1 # Convertir segundos a minutos y horas
2 print("== CONVERSION DE TIEMPO ==")
3
4 segundos = int(input("Ingresa los segundos: "))
5
6 minutos = segundos // 60
7 segundos_restantes = segundos % 60
8 horas = minutos // 60
9 minutos_restantes = minutos % 60
10
11 print(f"{segundos} segundos equivalen a:")
12 print(f"{horas} horas, {minutos_restantes} minutos y {
    segundos_restantes} segundos")

```

6.3 Ejemplo 3: Promedio de calificaciones

```

1 # Calcular promedio de tres calificaciones
2 print("== PROMEDIO DE CALIFICACIONES ==")
3
4 cal1 = int(input("Ingresa la calificaci n 1: "))
5 cal2 = int(input("Ingresa la calificaci n 2: "))
6 cal3 = int(input("Ingresa la calificaci n 3: "))
7
8 promedio = (cal1 + cal2 + cal3) / 3
9 promedio_entero = (cal1 + cal2 + cal3) // 3
10
11 print(f"El promedio exacto es: {promedio:.2f}")
12 print(f"El promedio entero es: {promedio_entero}")

```

7 Resumen y recomendaciones

7.1 Puntos clave

- `input()` siempre devuelve una cadena de texto
- `int()` convierte valores a n3meros enteros

- Es necesario usar `int()` con `input()` para operaciones matemáticas
- `print()` muestra información en la consola
- Los enteros en Python soportan operaciones aritméticas básicas
- La división con `/` devuelve flotante, `//` devuelve entero

7.2 Errores comunes a evitar

1. Olvidar convertir el input a entero:

```

1      # Incorrecto
2      num = input("N mero: ")
3      resultado = num + 5  # Error: TypeError
4
5      # Correcto
6      num = int(input("N mero: "))
7      resultado = num + 5
8

```

2. Intentar convertir texto no numérico con `int()`:

```

1      # Esto genera error
2      # numero = int("abc")
3

```

3. Confundir división flotante con división entera:

```

1      print(7 / 2)      # 3.5 (flotante)
2      print(7 // 2)     # 3   (entero)
3

```

8 Conclusión

Los comandos `input()`, `print()` y `int()` son herramientas fundamentales en Python que permiten la interacción con el usuario y el manejo de datos numéricos. Comprender su funcionamiento y saber combinar estos elementos es esencial para desarrollar programas que reciban información del usuario, la procesen y muestren resultados de manera efectiva.

Las operaciones con variables enteras proporcionan la base para realizar cálculos y resolver problemas computacionales, desde los más simples hasta los más complejos. Dominar estos conceptos es el primer paso para convertirse en un programador competente en Python.